

50 dbt Interview Questions & Answers

Fundamentals & Core Concepts

1. What is dbt and what problem does it solve?

Answer: dbt (data build tool) is a transformation tool that enables analytics engineers to version-control, test, and document SQL transformations in a modular, reusable way.

Problems it solves:

- **No version control:** SQL queries scattered in notebooks with no history
- **No testing:** No data quality validation before warehouse loads
- **No documentation:** "Where does this metric come from?" takes 2 hours
- **Code duplication:** Same aggregation logic in 5 different reports
- **Deployment friction:** Manual scripts, inconsistent environments

Example:

```
sql

-- Before dbt: Raw SQL file, no version control, no tests
SELECT user_id, COUNT(*) as order_count
FROM raw_orders
GROUP BY 1

-- With dbt: Versioned, documented, tested
SELECT user_id, COUNT(*) as order_count
FROM {{ ref('stg_orders') }}
GROUP BY 1
```

2. What's the difference between dbt Core and dbt Cloud?

Answer:

Feature	dbt Core	dbt Cloud
Cost	Free, open-source	\$1-5k+/month (pricing tiers)
Orchestration	None (use Airflow, Cron)	Built-in scheduling, continuous integration
IDE	Local IDE (VSCode)	Web-based IDE, Git integration
Version Control	Manual Git setup	Automatic Git sync on every commit
Environment Management	Manual (env_var config)	Built-in dev/staging/prod dbt environments
Notifications	Manual setup	Slack, email, webhook alerts built-in
dbt Cloud Features	N/A	Slim CI/CD, auto-docs, API, analytics

Use **dbt Core** if: Small team, existing orchestration (Airflow), cost-sensitive
 Use **dbt Cloud** if: Multi-team, need enterprise features, want managed service

3. What are the main files and directories in a dbt project?

Answer:

```

my_dbt_project/
├─ dbt_project.yml          # Project config (name, version,
profiles_dir)
├─ profiles.yml            # Database credentials (usually
~/.dbt/profiles.yml)
├─ models/
│   ├─ staging/            # dbt staging models
│   │   └─ stg_customers.sql
│   │   └─ stg_orders.sql
│   ├─ intermediate/      # Business logic layer
│   │   └─ int_customer_metrics.sql
│   └─ marts/             # Domain-specific outputs
│       ├─ finance/
│       │   └─ fct_revenue.sql
│       └─ marketing/
│           └─ fct_customer_segment.sql
├─ tests/                 # Custom test files
│   └─ data_tests/
│       └─ dbt_project_tests.py
├─ macros/               # Jinja2 reusable logic
│   └─ generate_schema_name.sql
│   └─ deduplicate.sql
├─ analysis/             # One-off analysis SQL (not deployed)
├─ data/                 # Seeds (static CSV files)
│   └─ country_codes.csv
├─ snapshots/            # Slowly changing dimensions
│   └─ dim_customers_snapshot.sql
└─ target/               # Compiled outputs (generated)
    ├─ compiled/
    │   └─ models/
    │       └─ staging/
    │           └─ stg_customers.sql
    └─ manifest.json      # Lineage, metadata

```

Key files:

- `dbt_project.yml`: Project name, vars, model paths
- `profiles.yml`: Connection details to warehouse
- `models/`: SQL files (main work)
- `tests/`: Data quality assertions

4. What is a dbt model?

Answer: A dbt model is a reusable unit of SQL logic (a SELECT statement) that produces

a table or view in your warehouse.

Example:

```
sql
-- models/staging/stg_orders.sql
SELECT
  order_id,
  customer_id,
  order_date,
  amount
FROM {{ source('raw', 'orders') }}
WHERE amount > 0
      AND order_date IS NOT NULL
```

Materialization (how dbt creates the model):

Type	Output	Use Case	Performance
View	SQL view (no storage)	Lightweight transformations	Slower (re-executes on every query)
Table	Actual table	Fact/dimension tables, heavy usage	Faster (pre-computed, indexed)
Incremental	Append-only table	Event logs, time-series (huge volumes)	Fastest (only process new rows)
Ephemeral	Temporary CTE	Intermediate calculations	Inline (not stored)
Dynamic table	Auto-refreshing table	Snowflake-specific, low latency	Real-time (auto-refresh)

5. What's the difference between source() and ref()?

Answer:

Function	Purpose	Points To	Example
source()	Ref raw data	External tables (raw zone, database)	{{ source('raw', 'orders') }}
ref()	Ref other models	dbt models only	{{ ref('stg_orders') }}

Why it matters:

- **Lineage tracking:** dbt builds dependency graph automatically
- **Refactoring safety:** Rename a model, all refs update automatically
- **Governance:** "Which models depend on the customer table?"

Example:

```
sql

-- models/staging/stg_orders.sql
SELECT * FROM {{ source('raw', 'orders') }}

-- models/intermediate/int_orders_enriched.sql
SELECT
  o.*,
  c.customer_name
FROM {{ ref('stg_orders') }} o           -- Model dependency
LEFT JOIN {{ ref('stg_customers') }} c   -- Model dependency
  ON o.customer_id = c.customer_id

-- dbt build output:
-- stg_orders → int_orders_enriched → downstream_marts
```

Sources definition (YAML):

```
yaml

# models/sources.yml
sources:
  - name: raw
    database: production
    schema: raw_zone
    tables:
      - name: orders
        description: Raw orders from Postgres
      - name: customers
        description: Raw customers from CRM
```

Models & Materialization

6. When would you use a view vs. a table vs. an incremental model?

Answer:

Use VIEW when:

- Transformation is simple (filter, rename columns)
- Table changes frequently (always latest data)
- Storage cost is critical
- Data volume is small (<1GB)

```
sql

-- models/staging/stg_customers.sql
{{ config(materialized='view') }}

SELECT
  id as customer_id,
  name as customer_name,
  email
FROM {{ source('crm', 'customers') }}
```

Use TABLE when:

- Complex aggregations (GROUP BY, JOINS, window functions)
- Heavy query traffic (>100 queries/day)
- Need indexing for fast lookups
- Data volume is medium (1GB-100GB)

```
sql

-- models/marts/fct_customer_metrics.sql
{{ config(materialized='table') }}

SELECT
  c.customer_id,
  c.customer_name,
  COUNT(o.order_id) as lifetime_orders,
  SUM(o.amount) as lifetime_value,
  MAX(o.order_date) as last_order_date
FROM {{ ref('stg_customers') }} c
LEFT JOIN {{ ref('stg_orders') }} o USING(customer_id)
GROUP BY 1, 2
```

Use INCREMENTAL when:

- Append-only data (new rows only, no updates)
- Massive volume (>1M rows/day)
- Need fresh data within SLA

- Cost optimization critical

```
sql

-- models/marts/fct_events.sql
{{ config(
    materialized='incremental',
    on_schema_change='sync_all_columns'
) }}

SELECT
    event_id,
    user_id,
    event_type,
    event_timestamp,
    properties
FROM {{ source('kinesis', 'raw_events') }}

{% if execute_macros and execute and flags.FULL_REFRESH is False %}
    WHERE event_timestamp > (SELECT MAX(event_timestamp) FROM {{ this }})
{% endif %}
```

Performance comparison (1M new rows/day):

- Full refresh (rebuild table): 5 min
- Incremental (append only): 30 sec
- View (compute on query): 30 sec per query × 100 queries = 50 min total

7. Explain incremental models and the dbt_internal_metamodel pattern.

Answer:

Incremental models process only new rows since last run, not full table rebuild.

How it works:

```
sql
```

```
-- models/fct_events.sql
{{ config(materialized='incremental') }}
```

SELECT

```
    event_id,
    user_id,
    event_timestamp,
    properties
FROM {{ source('events', 'raw_events') }}
```

-- This filter only runs after the first full refresh

```
{% if execute_macros and execute and not flags.FULL_REFRESH %}
    WHERE event_timestamp > (SELECT MAX(event_timestamp) FROM {{ this }})
{% endif %}
```

Key variables:

- `this`: Reference to the current model's target table
- `flags.FULL_REFRESH`: `dbt run --full-refresh` rebuilds from scratch
- `execute`: Jinja2 execute flag (determines if logic runs)

Example scenario:

Day 1: `dbt run --full-refresh`

- Processes all 10M historical events
- Creates table with 10M rows
- Takes 10 minutes

Day 2: `dbt run (normal)`

- Processes only events from last 24h (100k rows)
- Appends to existing table → 10.1M rows
- Takes 1 minute

Advanced: `dbt_internal_metamodel` (soft deletes)

If source data has updates (not just appends), use upsert logic:

sql


```
-- models/fct_orders_incremental.sql
{{ config(
    materialized='incremental',
    unique_key='order_id',
    on_schema_change='sync_all_columns'
) }}

SELECT
    order_id,
    customer_id,
    amount,
    status,
    updated_at
FROM {{ source('orders', 'raw_orders') }}

{% if execute_macros and execute and not flags.FULL_REFRESH %}
    WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{% endif %}
```

dbt automatically:

1. Inserts new rows (order_id not in table)
2. Updates existing rows (order_id exists, newer updated_at)
3. Deletes soft-deleted rows (status = 'deleted')

8. What's the difference between ephemeral and temporary materialization?

Answer:

Feature	Ephemeral	Temporary
Output	None (inlined as CTE)	Temporary table (session-scoped)
Visibility	Only to dependent models	Visible in schema
Persistence	Exists only during compile	Dropped after session ends
Performance	No I/O overhead	Single compilation pass
Use case	Shared logic, intermediate steps	Very large intermediate tables

Ephemeral example:

```
sql
```

```
-- models/intermediate/int_customer_base.sql
{{ config(materialized='ephemeral') }}

SELECT
    customer_id,
    customer_name,
    created_date,
    account_status
FROM {{ ref('stg_customers') }}
```

How dbt compiles it:

```
sql

-- Before dbt compile
-- models/marts/fct_customers.sql
WITH int_customer_base AS (
    SELECT ... -- Inlined CTE
    FROM {{ ref('int_customer_base') }}
)
SELECT * FROM int_customer_base

-- After dbt compile (ephemeral inlined)
WITH int_customer_base AS (
    SELECT customer_id, customer_name, created_date, account_status
    FROM stg_customers
)
SELECT * FROM int_customer_base
```

When to use ephemeral:

- Helper models used by 2+ downstream models
- Don't need to test/query independently
- Reduce schema clutter

Avoid ephemeral if:

- Large intermediate table (>100GB) — queries become slow with huge CTEs
- Multiple different downstream queries — better as table for parallel execution

Testing

9. What are the four main dbt tests and how do they work?

Answer:

dbt provides four **generic tests** (not Python-based):

1. Not Null

```
yaml

models:
  - name: customers
    columns:
      - name: customer_id
        tests:
          - not_null
```

Generated SQL:

```
sql

SELECT COUNT(*) as validation_errors
FROM customers
WHERE customer_id IS NULL
HAVING COUNT(*) > 0
```

Fails if: Any NULL values found

2. Unique

```
yaml

columns:
  - name: customer_id
    tests:
      - unique
```

Generated SQL:

```
sql

SELECT customer_id, COUNT(*) as n_records
FROM customers
GROUP BY customer_id
HAVING COUNT(*) > 1
```

Fails if: Duplicate values exist

3. Relationships (Foreign Key)

```
yaml

columns:
  - name: order_id
    tests:
      - relationships:
          to: ref('orders')
          field: order_id
```

Generated SQL:

```
sql

SELECT COUNT(*) as validation_errors
FROM {{ ref('orders') }}
WHERE order_id NOT IN (SELECT order_id FROM customers)
```

Fails if: Foreign key references non-existent parent records

4. Accepted Values

```
yaml

columns:
  - name: status
    tests:
      - accepted_values:
          values: ['active', 'inactive', 'pending']
```

Generated SQL:

```
sql

SELECT COUNT(*) as validation_errors
FROM customers
WHERE status NOT IN ('active', 'inactive', 'pending')
AND status IS NOT NULL
```

Fails if: Column contains values outside the whitelist

10. How do you write custom (singular) tests in dbt?

Answer:

Generic tests (YAML) handle 80% of cases. **Singular tests** (SQL files) handle complex logic.

Example: Custom test for revenue anomalies

```
sql

-- tests/assert_daily_revenue_not_null.sql
SELECT COUNT(*) as validation_errors
FROM {{ ref('fct_daily_revenue') }}
WHERE DATE(revenue_date) = CURRENT_DATE()
      AND total_revenue IS NULL
HAVING COUNT(*) > 0
```

How dbt runs it:

```
bash

dbt test --select assert_daily_revenue_not_null

# If query returns 0 rows → ✅ PASS
# If query returns >0 rows → ❌ FAIL (returns the error rows)
```

Advanced: Custom test with severity config

```
sql

-- tests/dbt_expect_revenue_growth.sql
{{ config(severity='warn') }} -- Don't fail pipeline, just warn

WITH daily_metrics AS (
  SELECT
    DATE(revenue_date) as date,
    SUM(amount) as daily_revenue,
    LAG(SUM(amount)) OVER (ORDER BY DATE(revenue_date)) as prior_day_revenue
  FROM {{ ref('fct_transactions') }}
  GROUP BY 1
)
SELECT *
FROM daily_metrics
WHERE daily_revenue < (prior_day_revenue * 0.5) -- Drop >50%
      AND DATE(date) != '2025-01-01' -- Exclude holidays
```

Run with severity levels:

```
bash
```

```
dbt test --severity error      # Only fail on errors
dbt test --severity warn      # Fail on warnings + errors
```

11. How do you test data freshness in dbt?

Answer:

Freshness checks ensure source tables update on schedule.

```
yaml
# models/sources.yml
sources:
  - name: raw
    database: production
    schema: postgres_raw
    tables:
      - name: customers
        description: Customer records from Postgres
        loaded_at_field: updated_at # Timestamp column to check
        freshness:
          warn_after: {count: 12, period: hour} # Warn if >12h old
          error_after: {count: 24, period: hour} # Fail if >24h old
```

How dbt checks freshness:

```
sql
-- dbt auto-generates this query
SELECT MAX(updated_at) as last_loaded
FROM production.postgres_raw.customers
```

Run freshness checks:

```
bash

dbt source freshness # Check all sources

# Output:
# raw.customers: PASS (last updated 2 hours ago)
# raw.orders: ERROR (last updated 48 hours ago) ❌
```

Use in CI/CD:

```
yaml
```

```
# dbt_project.yml
on-run-start:
  - "dbt source freshness" # Check before dbt run
```

If freshness check fails, CI pipeline stops and alerts team.

Sources & Staging

12. What is a dbt source and why use it?

Answer:

A **source** is a reference to raw external data (not a dbt model).

Why sources matter:

1. **Lineage tracking:** Understand raw data dependencies
2. **Documentation:** Who owns this raw table? What's the SLA?
3. **Governance:** Apply data quality tests to raw data
4. **Refactoring:** Change source table name once, all refs update

Example definition:

```
yaml
```

```
# models/sources.yml
sources:
  - name: postgres_raw          # Source namespace
    description: Postgres production database
    database: production
    schema: public

    tables:
      - name: customers
        description: Raw customers from Postgres
        columns:
          - name: id
            description: Customer ID (PK)
            tests:
              - unique
              - not_null
          - name: email
            description: Customer email
            tests:
              - unique

      - name: orders
        description: Raw orders
        loaded_at_field: created_at
        freshness:
          warn_after: {count: 6, period: hour}
        owner: analytics@company.com
```

Usage in models:

```
sql

SELECT * FROM {{ source('postgres_raw', 'customers') }}
```

Generated lineage:

```
postgres_raw.customers (external) → stg_customers (dbt model) →
customer_metrics (dbt model)
```

13. What's the staging layer and what transformations go there?

Answer:

Staging layer = First dbt models that ref raw sources.

Transformations in staging:

```
sql

-- models/staging/stg_customers.sql
{{ config(materialized='view') }}

SELECT
    id as customer_id,           -- Rename (consistency)
    LOWER(email) as email,       -- Standardize format
    UPPER(country_code) as country, -- Standardize
    created_at::DATE as signup_date, -- Type conversion
    COALESCE(status, 'unknown') as status -- Handle nulls
FROM {{ source('postgres_raw', 'customers') }}
WHERE id IS NOT NULL           -- Remove invalid records
    AND created_at >= '2020-01-01' -- Filter old data
```

Tests in staging:

```
yaml

models:
  - name: stg_customers
    tests:
      - dbt_expectations.expect_column_values_to_be_of_type:
          column_name: customer_id
          column_type: integer
      - dbt_expectations.expect_column_values_to_match_regex:
          column_name: email
          regex: "^[\\w\\.-]+@[\\w\\.-]+\\.\\w+$"
    columns:
      - name: customer_id
        tests:
          - unique
          - not_null
```

Staging best practices:

1. **One source per staging model** (stg_customers, stg_orders, not stg_crm_all)
 2. **Minimal logic** (rename, type cast, filter)
 3. **Materialize as VIEW** (lightweight)
 4. **Test heavily** (validate raw data quality)
-

Macros & Jinja2

14. What is a dbt macro and give an example.

Answer:

A **macro** is reusable Jinja2 template logic (like a function) in dbt.

Example: Deduplication macro

```
sql

-- macros/deduplicate.sql
{% macro deduplicate(relation, unique_key, order_by) %}
SELECT *
FROM {{ relation }}
QUALIFY ROW_NUMBER() OVER (PARTITION BY {{ unique_key }} ORDER BY {{ order_by }}
{% endmacro %}

-- Usage in a model
-- models/staging/stg_orders.sql
{{ config(materialized='view') }}

{{ deduplicate(
    source('raw', 'orders'),
    'order_id',
    'updated_at'
) }}
```

Generated SQL:

```
sql

SELECT *
FROM raw.orders
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC) =
```

15. What's the difference between a macro and a model?

Answer:

Aspect	Macro	Model
Input	Arguments (relation, column names)	Source/ref (tables, views)
Output	SQL code (templated)	Database object (table/view)
Execution	Compiles Jinja2 → SQL	Executes SQL in warehouse
Purpose	Reusable logic template	Data transformation
Example	<code>deduplicate(table, key)</code>	<code>SELECT * FROM stg_orders</code>

Macro:

```
sql

{% macro deduplicate(table, key) %}
    SELECT *
    FROM {{ table }}
    QUALIFY ROW_NUMBER() OVER (PARTITION BY {{ key }} ORDER BY 1 DESC) = 1
{% endmacro %}

-- Compiles to SQL, doesn't execute
```

Model:

```
sql

SELECT * FROM {{ source('raw', 'orders') }}

-- Compiles to SQL AND executes in warehouse
```

16. Write a macro that generates a series of date columns.

Answer:

```
sql
```

```

-- macros/generate_dates.sql
{% macro generate_dates(start_date, end_date, grain='day') %}
    {%- if grain == 'day' -%}
        SELECT CAST(generated_date AS DATE) as date
        FROM GENERATE_SERIES({{ dbt.string_literal(start_date) }}, {{ dbt.string_literal(end_date) }})
    {%- elif grain == 'month' -%}
        WITH date_spine AS (
            SELECT CAST(DATE_TRUNC('month', generated_date) AS DATE) as date
            FROM GENERATE_SERIES({{ dbt.string_literal(start_date) }}, {{ dbt.string_literal(end_date) }})
        )
        SELECT DISTINCT date FROM date_spine
    {%- endif -%}
{% endmacro %}

-- Usage in a model
-- models/marts/date_spine.sql
{{ config(materialized='table') }}

{{ generate_dates('2020-01-01', '2026-12-31', 'day') }}

```

Generated SQL (Postgres):

```

sql

SELECT CAST(generated_date AS DATE) as date
FROM GENERATE_SERIES('2020-01-01'::DATE, '2026-12-31'::DATE, INTERVAL '1 day')

```

17. Explain Jinja2 conditionals and loops in dbt.

Answer:

Conditionals (if/else):

```

sql

```

```
-- models/fct_orders.sql
{{ config(
    materialized = 'incremental',
    unique_key = 'order_id'
) }}

SELECT
    order_id,
    customer_id,
    amount,
    {% if target.type == 'snowflake' %}
        DATEDIFF(DAY, order_date, CURRENT_DATE()) as days_since_order
    {% elif target.type == 'postgres' %}
        CURRENT_DATE - order_date as days_since_order
    {% else %}
        DATE_DIFF(CURRENT_DATE(), order_date, DAY) as days_since_order
    {% endif %}
FROM {{ ref('stg_orders') }}

{% if execute_macros and execute and not flags.FULL_REFRESH %}
    WHERE order_date > (SELECT MAX(order_date) FROM {{ this }})
{% endif %}
```

Loops (for):

```
sql

-- macros/generate_columns.sql
{% macro generate_columns(column_list) %}
    SELECT
        order_id,
        {% for col in column_list %}
            {{ col }} as {{ col }}_value
            {% if not loop.last %}, {% endif %}
        {% endfor %}
    FROM orders
{% endmacro %}

-- Usage
-- models/orders_with_cols.sql
{{ generate_columns(['amount', 'quantity', 'discount']) }}
```

Generated SQL:

```
sql
```

```
SELECT
  order_id,
  amount as amount_value,
  quantity as quantity_value,
  discount as discount_value
FROM orders
```

Key Jinja2 variables:

- `loop.first`: Is this the first iteration?
 - `loop.last`: Is this the last iteration?
 - `loop.index`: 1-based iteration counter
 - `loop.index0`: 0-based iteration counter
-

Variables & Configurations

18. How do you use dbt variables?

Answer:

Variables = Runtime parameters for dbt models.

Definition (dbt_project.yml):

```
yaml

vars:
  my_bucket: "s3://analytics-bucket"
  start_date: "2020-01-01"
  is_production: false
  region_filter: ['US', 'CA', 'MX']
```

Usage in models:

```
sql

-- models/staging/stg_orders.sql
SELECT *
FROM {{ source('raw', 'orders') }}
WHERE order_date >= {{ var('start_date') }}
      AND region IN ({{ var('region_filter') | join(', ') }})
```

Override at runtime:

```
bash
```

```
dbt run --vars '{"start_date": "2023-01-01", "is_production": true}'
```

Conditional logic based on environment:

```
yaml
# dbt_project.yml
vars:
  env: "{{ env_var('DBT_ENV', 'dev') }}" # Read from environment variable
```

Use cases:

- Date ranges (reprocess last 30 days)
- Feature flags (enable/disable models)
- Environment-specific schemas
- Regional filters

19. What's the difference between dbt_project.yml and profiles.yml?

Answer:

File	Location	Purpose	Contains
dbt_project.yml	Project root	Project configuration	Model paths, vars, macro paths, seeds_paths
profiles.yml	~/.dbt/ (home dir)	Warehouse credentials	Database type, host, port, username, password

dbt_project.yml example:

```
yaml
```

```
name: 'my_analytics'
version: '1.0.0'
config-version: 2

profile: 'my_profile' # Reference to profiles.yml
model-paths: ["models"]
test-paths: ["tests"]
macro-paths: ["macros"]
seed-paths: ["data"]

models:
  my_analytics:
    staging:
      materialized: view
    marts:
      materialized: table
      pre-hook: "{{ log('Running marts', info=true) }}"
      post-hook: "GRANT SELECT ON {{ this }} TO ROLE analyst"

vars:
  start_date: '2020-01-01'
  bucket_name: 'analytics-bucket'
```

profiles.yml example (never commit to Git!):

yaml


```
my_profile:
  outputs:
    dev:
      type: redshift
      host: dev-cluster.redshift.amazonaws.com
      user: "{{ env_var('REDSHIFT_USER') }}"
      pass: "{{ env_var('REDSHIFT_PASSWORD') }}"
      port: 5439
      dbname: analytics_dev
      schema: dbt_nitin
      threads: 4
      keepalives_idle: 0

    prod:
      type: redshift
      host: prod-cluster.redshift.amazonaws.com
      user: "{{ env_var('REDSHIFT_USER') }}"
      pass: "{{ env_var('REDSHIFT_PASSWORD') }}"
      port: 5439
      dbname: analytics_prod
      schema: analytics_marts
      threads: 8

target: dev # Default target
```

Never commit profiles.yml! Use:

```
bash

git update-index --skip-worktree ~/.dbt/profiles.yml
```

20. What are pre-hooks and post-hooks in dbt?

Answer:

Hooks = SQL that runs before/after model creation.

Pre-hook: Runs BEFORE model SQL **Post-hook:** Runs AFTER model SQL

```
yaml
```

```
# dbt_project.yml
models:
  my_analytics:
    staging:
      pre-hook:
        - "{{ log('Starting staging models', info=true) }}"
        - "CREATE TEMP TABLE {{ this.schema }}.tmp_data AS SELECT * FROM raw"

      post-hook:
        - "GRANT SELECT ON {{ this }} TO ROLE analyst"
        - "INSERT INTO {{ this.schema }}.model_audit (model_name, dbt_updated_at, dbt_run_id) VALUES ('{{ this.name }}', CURRENT_TIMESTAMP, '{{ run_id }}')"
```

Per-model hooks (within SQL file):

```
sql

-- models/staging/stg_orders.sql
{{ config(
  pre-hook="ALTER TABLE raw.orders SET TBLPROPERTIES ('LAST_MODIFIED_TIME'=CURRENT_TIMESTAMP)",
  post-hook="CALL REFRESH_TABLE('{{ this }}')",
) }}

SELECT * FROM {{ source('raw', 'orders') }}
```

Common use cases:

1. **Pre-hook:** Staging tables, temp data
2. **Post-hook:** Permissions, table stats, audit logging

```
sql

post-hook: [
  "ANALYZE TABLE {{ this }} COMPUTE STATISTICS", -- Update table stats
  "GRANT SELECT ON {{ this }} TO ROLE analyst",   -- Set permissions
  "INSERT INTO dbt_audit (model, run_time) VALUES ('{{ this.name }}', CURRENT_TIMESTAMP)",
]
```

Performance & Optimization

21. How do you optimize slow dbt runs?

Answer:

Step 1: Find bottleneck

```
bash
```

```
dbt debug --debug > dbt.log 2>&1  
dbt run --debug --select fct_revenue # Run single model
```

Look for:

- Model compile time (>5 min = too much Jinja2)
- Model execute time (>10 min = inefficient SQL)
- I/O time (reading/writing large tables)

Step 2: Profile execution

```
bash
```

```
# See model runtimes  
dbt run --debug | grep -i "completed"
```

Step 3: Optimize SQL

Before (slow):

```
sql
```

```
-- Full table join on every row  
SELECT o.*, c.customer_name  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
WHERE o.order_date >= '2025-01-01'
```

After (fast):

```
sql
```

```
-- Filter before join  
SELECT o.*, c.customer_name  
FROM orders o  
INNER JOIN customers c ON o.customer_id = c.customer_id -- INNER vs LEFT  
WHERE o.order_date >= '2025-01-01' -- Pushed down filter  
      AND c.status = 'active' -- Filter on dimension
```

Step 4: Use incremental models

```
sql
```

```
-- Instead of rebuilding 100M-row table daily
{{ config(materialized='incremental') }}

SELECT *
FROM {{ source('events', 'raw_events') }}

{% if execute_macros and execute and not flags.FULL_REFRESH %}
    WHERE event_timestamp > (SELECT MAX(event_timestamp) FROM {{ this }})
{% endif %}
```

Step 5: Optimize materialization

```
yaml

# Don't materialize intermediate models as tables
models:
  intermediate:
    materialized: ephemeral # Inline as CTE
  marts:
    materialized: table      # Store final output
```

Step 6: Parallelize with threads

```
yaml

# profiles.yml
my_analytics:
  outputs:
    prod:
      threads: 8 # Use 8 parallel threads
```

Benchmark:

- 10 staging models: 5 min (1 thread) → 1 min (8 threads)

22. How do you handle large table rebuilds (>100GB)?

Answer:

Option 1: Incremental + Full Refresh

```
sql
```

```

{{ config(
  materialized='incremental',
  unique_key='event_id'
) }}

SELECT *
FROM {{ source('events', 'raw_events') }}

{% if execute_macros and execute and not flags.FULL_REFRESH %}
  WHERE event_date > (SELECT MAX(event_date) FROM {{ this }})
{% endif %}

```

Daily run (incremental):

```

bash

dbt run --select fct_events # 1 min (append 100k rows)

```

Monthly full refresh (scheduled):

```

bash

dbt run --full-refresh --select fct_events # 20 min (rebuild 100M rows)

```

Option 2: External table (don't materialize)

```

sql

{{ config(
  materialized='table',
  external_location='s3://bucket/events/'
) }}

SELECT *
FROM {{ source('events', 'raw_events_parquet') }}

```

Option 3: Snowflake dynamic tables (auto-refresh):

```

sql

{{ config(materialized='dynamic_table') }}

SELECT *
FROM {{ ref('stg_events') }}

```

Option 4: Snapshot data in batches

sql

```
-- Save last 7 days to table, older to archive
{{ config(materialized='incremental') }}

SELECT *
FROM {{ source('events', 'raw_events') }}
WHERE event_date >= CURRENT_DATE() - 7

{% if execute_macros and execute and not flags.FULL_REFRESH %}
    AND event_date > (SELECT MAX(event_date) FROM {{ this }})
{% endif %}
```

Advanced Topics

23. What is a dbt snapshot and when do you use it?

Answer:

A **snapshot** captures the state of a table at a point in time, tracking changes over time (Slowly Changing Dimension Type 2).

Use case: Track customer attributes changes

```
2025-01-01: Customer #123 → Status='Active', Segment='Premium'
2025-02-15: Customer #123 → Status='Active', Segment='Gold'   (updated)
2025-03-20: Customer #123 → Status='Inactive', Segment='Churned' (updated)
```

Snapshot definition:

sql

```
-- snapshots/customers_snapshot.sql
{% snapshot customers_snapshot %}
{{
    config(
        target_schema='snapshots',
        unique_key='customer_id',
        strategy='timestamp',
        updated_at='updated_at', -- Column that changes when row updates
    )
}}

SELECT
    customer_id,
    customer_name,
    status,
    segment,
    created_at,
    updated_at
FROM {{ source('crm', 'customers') }}

{% endsnapshot %}
```

Generated snapshot table:

customer_id	customer_name	status	segment	dbt_scd_id	dbt_updated_at	dbt_valid_from	dbt_valid_to
123	Alice	Active	Premium	abc123	2025-01-01	2025-01-01	2025-02-15
123	Alice	Active	Gold	def456	2025-02-15	2025-02-15	2025-03-20
123	Alice	Inactive	Churned	ghi789	2025-03-20	2025-03-20	NULL

Query current state:

```
sql

SELECT * FROM snapshots.customers_snapshot
WHERE dbt_valid_to IS NULL
```

Track historical state:

```
sql
```

```
SELECT *
FROM snapshots.customers_snapshot
WHERE customer_id = 123
      AND dbt_valid_from <= '2025-02-01'
      AND dbt_valid_to > '2025-02-01'
-- Returns: Active/Premium state on Feb 1
```

24. Explain dbt dependencies and ref() resolution order.

Answer:

dbt builds a **dependency graph** (DAG) and runs models in order.

Example:

```
raw_orders (source)
  ↓
stg_orders (ref source)
  ↓
int_orders_enriched (ref stg_orders)
  ↓
fct_orders (ref int_orders_enriched)
  ↓
revenue_dashboard (ref fct_orders)
```

Run all models:

```
bash

dbt run # Executes in dependency order (top to bottom)
```

Run specific model and dependencies:

```
bash

dbt run --select fct_orders # Run fct_orders only

dbt run --select +fct_orders # Run stg_orders → fct_orders

dbt run --select fct_orders+ # Run fct_orders → revenue_dashboard

dbt run --select +fct_orders+ # Run both directions
```

Run models with specific tag:

See dependency graph:

```
bash

dbt docs generate  # View as interactive graph
```

Circular dependency error:

```
dbt shows error if:
  stg_orders → refs('stg_customers')
  stg_customers → refs('stg_orders')  ← CIRCULAR!
```

Solution: Use **ephemeral model** to share logic without circular ref.

25. What is dbt test and how do you run tests selectively?

Answer:

Tests validate data quality before pushing to production.

Run all tests:

```
bash

dbt test  # Runs all generic + singular tests
```

Run specific test:

```
bash

dbt test --select stg_customers_unique_id  # Run one test

dbt test --select customers  # Run all tests on 'customers' model

dbt test --select source:raw.orders  # Test specific source
```

Run tests by type:

```
bash
```

```
dbt test --schema # Only run schema tests (not_null, unique, etc.)
```

```
dbt test --data # Only run data tests (singular tests)
```

Run tests on model and downstream:

```
bash
```

```
dbt test --select +fct_orders # Test dependencies first
```

```
dbt test --select fct_orders+ # Test fct_orders and what uses it
```

Run tests with severity filtering:

```
bash
```

```
dbt test --select severity:error # Only fail on errors
```

```
dbt test --select severity:warn # Fail on warnings + errors
```

Fail fast (stop on first failure):

```
bash
```

```
dbt test --fail-fast
```

Test output:

```
results:
```

- test_name: dbt_expectations_expect_column_values_to_not_be_null
status: pass
model: stg_customers
- test_name: dbt_expectations_expect_column_values_to_be_unique
status: fail ❌
model: stg_orders
rows_with_issue: 1234

26. How do you handle schema changes in dbt models?

Answer:

When upstream source schema changes (new column, renamed column, dropped column), dbt handles it with `on_schema_change`.

Example: Source adds new column

```
sql

-- Source table adds: phone_number column
-- dbt incremental model needs to handle it
{{ config(
    materialized='incremental',
    on_schema_change='sync_all_columns' # Auto-add new columns
) }}

SELECT *
FROM {{ source('crm', 'customers') }}

{% if execute_macros and execute and not flags.FULL_REFRESH %}
    WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{% endif %}
```

Options for `on_schema_change`:

Option	Behavior	Use Case
<code>ignore</code>	Do nothing (old columns only)	Backwards compatible
<code>fail</code>	Raise error, stop	Strict schema enforcement
<code>sync_all_columns</code>	Add new columns, drop old	Auto-adapt to schema
<code>append_new_columns</code>	Only add new columns	Preserve history

Best practice:

```
yaml

# dbt_project.yml
models:
  staging:
    +on_schema_change: 'append_new_columns' # Conservative
```

27. What is a dbt exposure and why use it?

Answer:

An **exposure** documents downstream usage of dbt models (dashboards, reports, ML models, APIs).

Why it matters:

- Impact analysis: "If I change this model, what breaks?"
- Discoverability: Find all dashboards using a metric
- Ownership: Who owns this dashboard? Who can change the underlying data?

Example:

```
yaml

# models/exposures.yml
exposures:
  - name: finance_revenue_dashboard
    type: dashboard
    maturity: production
    owner:
      name: Finance Analytics Team
      email: finance-analytics@company.com
    description: Daily revenue by product line
    depends_on:
      - ref('fct_daily_revenue')
      - ref('dim_products')
    url: https://tableau.company.com/views/revenue-dashboard
    tags: [daily, critical]
    meta:
      sla_minutes: 60

  - name: customer_churn_ml_model
    type: ml
    maturity: beta
    owner:
      name: Data Science Team
    depends_on:
      - ref('fct_customer_features')
      - ref('dim_customer_attributes')
    description: Predicts churn probability for next 30 days
```

Query exposures in dbt docs:

```
bash

dbt docs generate
# dbt/index.html → navigate to Exposure → see all dependencies
```

Impact analysis:

```
bash
```

```
# "If I change fct_customer_features, what breaks?"  
dbt show --select fct_customer_features+  
# Shows: fct_customer_features → customer_churn_ml_model (exposure)
```

Best Practices & Real-World Scenarios

28. What's the dbt project structure you'd recommend for a large team?

Answer:

```
my_dbt_project/
├─ dbt_project.yml
├─ README.md
|
├─ models/
|   ├─ staging/
|   |   ├─ _stg_models.yml          # Docs, tests for all staging
|   |   ├─ stg_salesforce__accounts.sql
|   |   ├─ stg_salesforce__opportunities.sql
|   |   ├─ stg_postgres__users.sql
|   |   └─ stg_postgres__orders.sql
|   |
|   ├─ intermediate/
|   |   ├─ _int_models.yml
|   |   ├─ int_customers__with_revenue.sql
|   |   └─ int_orders__with_customer_data.sql
|   |
|   └─ marts/
|       ├─ finance/
|       |   ├─ _finance_models.yml
|       |   ├─ _finance_sources.yml
|       |   └─ fct_monthly_revenue.sql
|       |
|       ├─ marketing/
|       |   ├─ _marketing_models.yml
|       |   └─ dim_customer_segment.sql
|       |
|       └─ operations/
|           ├─ _operations_models.yml
|           └─ fct_support_tickets.sql
|
|   └─ _sources.yml                # All sources in one file
|
├─ tests/
|   ├─ generic/
|   |   └─ assert_revenue_not_negative.sql
|   └─ data/
|       └─ test_monthly_revenue_not_null.sql
|
├─ macros/
|   ├─ _macros.yml                # Macro documentation
|   ├─ generate_date_spine.sql
|   └─ deduplicate_by_timestamp.sql
|
├─ snapshots/
|   └─ dim_customers_snapshot.sql
```

```

|
├── seeds/
|   ├── _seeds.yml
|   └── country_codes.csv
|
├── analysis/
|   └── revenue_analysis_jan2025.sql    # One-off analysis
|
└── docs/
    ├── fields.md                      # Custom docs
    └── architecture.md

```

Why this structure?

1. **By domain (finance, marketing):** Teams own their mart
2. **Staging separate:** All raw data cleaning in one place
3. **Naming convention:** `(stg_source__domain)`, `(int_domain__concept)`, `(fct_/dim_)`
4. **Tests colocated:** `(_stg_models.yml)` alongside models
5. **Docs at top:** One source of truth for all docs

29. How do you handle timezone issues in dbt?

Answer:

Problem: Order created at "2025-06-16 14:30:00 UTC" means different times in different zones.

Solution: Store everything in UTC, convert on query

```

sql

-- models/staging/stg_orders.sql
SELECT
  order_id,
  customer_id,
  -- Store created_at as UTC always
  created_at AT TIME ZONE 'UTC' as created_at_utc,
  -- Extract date in UTC (not user timezone)
  DATE(created_at AT TIME ZONE 'UTC') as created_date_utc
FROM {{ source('orders', 'raw_orders') }}

```

Convert to local timezone on demand:

```

sql

```

```
-- models/marts/fct_orders.sql
SELECT
  order_id,
  created_at_utc,
  -- User can convert to their timezone in downstream query
  created_at_utc AT TIME ZONE 'America/Los_Angeles' as created_at_pst,
  created_at_utc AT TIME ZONE 'Europe/London' as created_at_gmt
FROM {{ ref('stg_orders') }}
```

In Tableau/BI tool: Set user timezone preference, Tableau converts display

Best practices:

1. Always store as UTC
2. Document timezone assumptions in YML
3. Add tests to verify no mixed timezones
4. Avoid timezone logic in dbt models (push to BI layer)

```
yaml
columns:
  - name: created_at_utc
    description: Order creation time in UTC. ALL TIMES STORED IN UTC.
```

30. How do you version dbt and manage breaking changes?

Answer:

dbt version in profiles.yml:

```
yaml
config-version: 2 # dbt 1.0+
require-dbt-version: ['>=1.4.0', '<1.6.0'] # Require dbt 1.4-1.5
```

Track breaking changes:

dbt Version	Breaking Change	Mitigation
1.3 → 1.4	<code>ref()</code> resolved differently	Update tests, monitor compile time
1.4 → 1.5	<code>on_schema_change</code> default changed	Explicitly set in config
1.5 → 1.6	dbt Cloud API changed	Update CI/CD integration

Test before upgrading:

```
bash

# In dev environment
dbt --version # Current: 1.4.2
dbt deps
dbt parse # Check for parse errors
dbt test # Run all tests
# If ✅, safe to upgrade production
```

Upgrade path:

```
bash

dbt run --dbt-version 1.5.0 # Test with new version

# If OK:
pip install dbt-redshift==1.5.0
# Update dbt_project.yml
# Re-test everything
```

31. How do you handle PII and data governance in dbt?

Answer:

1. Mask PII in staging:

```
sql

-- models/staging/stg_customers.sql
SELECT
    customer_id,
    CASE WHEN {{ is_test() }}          -- Hide PII in dev/test
        THEN 'REDACTED'
        ELSE SUBSTRING(email, 1, 3) || '***@***' -- Mask in prod
    END as email_masked,
    CASE WHEN {{ is_test() }}
        THEN 'REDACTED'
        ELSE phone_number -- Full phone in prod
    END as phone
FROM {{ source('crm', 'customers') }}
```

2. Tag sensitive data:

```
yaml
```

```
models:
  - name: stg_customers
    meta:
      owner: privacy-team@company.com
      contains_pii: true
      data_classification: confidential
    columns:
      - name: email
        meta:
          pii: email
      - name: ssn
        meta:
          pii: ssn
          column_type: personally_identifiable_info
```

3. Restrict access by role:

```
sql
-- Post-hook: Grant selective access
post-hook: >
  GRANT SELECT ON {{ this }} TO ROLE analyst;
  GRANT SELECT (customer_id, customer_name) ON {{ this }} TO ROLE intern;
```

4. Audit access:

```
sql
-- Log who queries PII
post-hook: >
  INSERT INTO audit.table_access (table_name, accessed_by, accessed_at)
  VALUES ('{{ this }}', CURRENT_USER(), CURRENT_TIMESTAMP())
```

5. Auto-mask in BI tools: Use dbt metadata + BI tool's field-level security to auto-hide PII columns from unauthorized users.

32. How do you implement slowly changing dimensions (SCD Type 2) in dbt?

Answer:

Option 1: dbt snapshots (simplest):

```
sql
```

```

-- snapshots/dim_customers_scd2.sql
{% snapshot dim_customers_scd2 %}
  {{
    config(
      target_schema='snapshots',
      unique_key='customer_id',
      strategy='timestamp',
      updated_at='updated_at'
    )
  }}

  SELECT
    customer_id,
    customer_name,
    segment,
    country,
    created_at,
    updated_at
  FROM {{ source('crm', 'customers') }}

{% endsnapshot %}

```

Output:

customer_id	segment	dbt_valid_from	dbt_valid_to	is_current
1	Premium	2025-01-01	2025-03-15	FALSE
1	Gold	2025-03-15	NULL	TRUE

Option 2: Manual SCD logic:

sql

```

-- models/marts/dim_customers_scd2.sql
{{ config(materialized='incremental', unique_key='customer_id') }}

WITH new_rows AS (
    SELECT
        customer_id,
        segment,
        updated_at
    FROM {{ source('crm', 'customers') }}
    {% if execute_macros and execute and not flags.FULL_REFRESH %}
        WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
    {% endif %}
),

existing_rows AS (
    SELECT * FROM {{ this }}
    WHERE is_current = TRUE
),

changed_rows AS (
    SELECT
        e.customer_id,
        NEW.updated_at,
        e.dbt_valid_to,
        FALSE as is_current
    FROM existing_rows e
    INNER JOIN new_rows NEW ON e.customer_id = NEW.customer_id
        AND e.segment != NEW.segment -- Dimension changed
)

SELECT
    customer_id,
    segment,
    CURRENT_TIMESTAMP as dbt_valid_from,
    NULL as dbt_valid_to,
    TRUE as is_current
FROM new_rows

UNION ALL

SELECT * FROM changed_rows

```

33. How do you test dbt models locally without running the full pipeline?

Answer:

1. Parse-only (fastest):

```
bash

dbt parse # Check for syntax errors, no SQL execution
```

2. Compile-only:

```
bash

dbt compile --select stg_customers # Compile Jinja2 → SQL, no execution
# Look in target/compiled/ for generated SQL
```

3. Run single model:

```
bash

dbt run --select stg_customers # Execute only this model
```

4. Dry-run (compile + explain, no write):

```
bash

dbt run-operation some_macro --args '{"key": "value"}' # Test macro
```

5. Test without full dataset:

```
sql

-- Add LIMIT for dev testing
{{ config(
  pre-hook="SELECT 1" -- Check if table exists
) }}

SELECT * FROM {{ source('raw', 'orders') }}
LIMIT 1000 -- Test with sample data
```

6. Use dbt Cloud IDE (easiest):

- Write SQL → Preview results → Commit
-

34. How do you handle late-arriving dimensions in dbt?

Answer:

Late-arriving dimension = Dimension row arrives after fact row that references it.

Example:

Day 1: Order #100 created with customer_id=5
Day 5: Customer #5 record finally lands in CRM

Fact table has order, but dimension has NULL customer!

Solution 1: Deferred fact load:

```
sql

-- Load facts only after dimensions exist
-- Schedule: Dimensions at 8pm, Facts at 9pm
-- Use task orchestration (Airflow, dbt Cloud) to enforce order
```

Solution 2: Nullable foreign key + fallback:

```
sql

-- models/marts/fct_orders.sql
SELECT
  o.order_id,
  COALESCE(c.customer_id, 'UNKNOWN') as customer_id,
  o.amount,
  COALESCE(c.segment, 'UNMAPPED') as customer_segment
FROM {{ ref('stg_orders') }} o
LEFT JOIN {{ ref('dim_customers') }} c -- LEFT JOIN for missing dimensions
  ON o.customer_id = c.customer_id
```

Solution 3: Late-arriving fact retry:

```
sql
```

```
-- models/marts/fct_orders_with_retry.sql
{{ config(
  materialized='incremental',
  unique_key='order_id'
) }}

WITH latest_orders AS (
  SELECT *
  FROM {{ ref('stg_orders') }}
  {% if execute_macros and execute and not flags.FULL_REFRESH %}
    WHERE order_date >= CURRENT_DATE() - 3 -- Look back 3 days for late arrivals
  {% endif %}
)

SELECT
  o.order_id,
  c.customer_id,
  o.amount
FROM latest_orders o
LEFT JOIN {{ ref('dim_customers') }} c ON o.customer_id = c.customer_id
```

Schedule dimension load before fact load (best practice).

CI/CD & Deployment

35. How do you set up dbt CI/CD with GitHub/GitLab?

Answer:

dbt Cloud + GitHub integration (easiest):

1. Connect dbt Cloud to GitHub repo
2. Configure webhook on PR creation
3. dbt Cloud automatically runs `dbt test` on every PR

yaml

```
# In dbt Cloud:
- Trigger: Pull request created/updated
- Run: dbt parse → dbt build (models + tests)
- Pass/Fail: Comment on PR with results
```

GitHub Actions (self-managed):

yaml

```

# .github/workflows/dbt-ci.yml
name: dbt CI

on:
  pull_request:
    branches: [main, develop]

jobs:
  dbt:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v2

      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: '3.9'

      - name: Install dbt
        run: pip install dbt-redshift

      - name: Authenticate
        run: |
          mkdir -p ~/.dbt
          echo '${{ secrets.DBT_PROFILES_YML }}' > ~/.dbt/profiles.yml

      - name: dbt parse
        run: dbt parse

      - name: dbt test (models changed in PR)
        run: dbt test --select state:modified+ --state prod-state

      - name: Comment PR with results
        if: always()
        uses: actions/github-script@v6
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '✅ dbt tests passed'
            })

```

Best practice CI/CD flow:

Code change on branch

↓

PR created → GitHub webhook

↓

dbt Cloud/Actions runs

- |─ dbt parse (syntax check)
- |─ dbt test --select state:modified+ (only changed models + downstream)
- |─ dbt snapshot (capture latest state)
- └─ dbt build (models + tests)

↓

All tests pass → Approve PR

↓

Merge to main

↓

Deploy to production (scheduled job runs full pipeline)

36. What's slim CI in dbt Cloud?

Answer:

Slim CI = Only test/run models that changed in the PR, plus their downstream dependencies.

Example:

dbt graph has 200 models

You changed: stg_orders.sql

Without slim CI:

dbt test → Runs tests on all 200 models (30 min) ❌

With slim CI:

dbt test --select state:modified+ --state prod-state

→ Runs tests on: stg_orders + all downstream models (int_orders, fct_orders, revenue_dashboard)

→ 2 min ✅

How to set up:

1. In dbt Cloud, enable "Slim CI"
2. dbt Cloud automatically saves `manifest.json` from last prod run
3. Compare current PR manifest to prod manifest
4. Identify changed models

5. Run tests on changed + downstream only

Command:

```
bash

dbt build --select state:modified+ --state /path/to/prod/manifest.json
```

Variables for slim CI:

- `state:new`: Models added in current run
 - `state:modified`: Models changed in current run
 - `state:modified+`: Changed + all downstream
-

37. How do you deploy dbt to production?

Answer:

Option 1: dbt Cloud (managed):

Setup:

1. Connect GitHub repo to dbt Cloud
2. Set up production credentials in dbt Cloud
3. Create scheduled job: "Run full pipeline daily at 2am"
4. Job runs: dbt deps → dbt seed → dbt build → dbt test

Monitoring:

- dbt Cloud dashboard shows job status
- Slack alerts on failure
- Email notifications

Option 2: Airflow (self-managed):

```
python
```

```
# airflow/dags/dbt_pipeline.py
from airflow import DAG
from cosmos import DbtTaskGroup

with DAG(dag_id='dbt_analytics', schedule_interval='0 2 * * *') as dag:
    dbt_tasks = DbtTaskGroup(
        group_id='dbt',
        project_dir='/path/to/dbt',
        profiles_config_dir=~/.dbt',
        dbt_cmd_args=['--select', 'path:models/staging+'],
        dbt_executable='/venv/bin/dbt'
    )
```

Option 3: GitHub Actions + S3 + Cron:

```
yaml
# .github/workflows/dbt-deploy.yml
name: dbt Deploy to Production

on:
  schedule:
    - cron: '0 2 * * *' # Daily at 2am

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run dbt
        run: dbt run --profiles-dir ~/.dbt --target prod
      - name: Upload artifacts
        run: aws s3 cp target/ s3://analytics-bucket/dbt-artifacts/
```

Deployment checklist:

- ☒ All tests pass
- ☒ Code reviewed (approved PR)
- ☒ Merged to main
- ☒ dbt seed data updated
- ☒ Credentials rotated
- ☒ Backup of current tables taken
- ☒ dbt docs generated
- ☒ Monitoring/alerts configured

Real-World Problem Solving

38. How do you handle a model that breaks in production?

Answer:

Scenario: Daily revenue model fails, dashboard blank

Step 1: Immediate response (5 min):

```
bash

# Check dbt Cloud logs
dbt logs --last run #123

# Find error:
# "Error: column 'price' does not exist"
```

Step 2: Root cause:

```
sql

-- Source table renamed column: 'price' → 'unit_price'
-- But dbt model still references 'price'
```

Step 3: Hotfix:

```
sql

-- Quick fix: Rename column in source
ALTER TABLE raw.orders RENAME COLUMN unit_price TO price;

-- Or: Update dbt model
SELECT
  order_id,
  unit_price as price, -- Alias old name
  ...
FROM {{ source('raw', 'orders') }}
```

Step 4: Deploy hotfix:

```
bash
```

```
git checkout -b hotfix/revenue-model-price-column
# Edit model
git commit -m "Fix: use unit_price instead of price"
git push
# Create PR, merge to main
dbt run --target prod # Re-run pipeline
```

Step 5: Prevent future issues:

```
yaml

# Add test for column existence
- name: price
  tests:
    - dbt_expectations.expect_column_to_exist
```

Post-incident:

- Document what happened
 - Add dbt freshness check (catch late arrivals)
 - Add schema validation tests
 - Notify source data team
-

39. How do you handle data discrepancies between dbt and source system?

Answer:

Scenario: Revenue in dbt = \$1M, Revenue in Salesforce = \$1.2M

Step 1: Identify where the discrepancy:

```
sql

-- Compare row counts
SELECT COUNT(*) FROM salesforce_raw.orders -- 50k rows
SELECT COUNT(*) FROM dbt.stg_orders        -- 48k rows
-- Missing 2k orders

-- Check for filtering
SELECT COUNT(*) FROM salesforce_raw.orders WHERE amount <= 0 -- 2k rows
-- AH! Negative amounts exist in raw, dbt filters them out
```

Step 2: Document assumption:

```
sql
```

```
-- models/staging/stg_orders.sql
SELECT *
FROM {{ source('salesforce', 'orders') }}
WHERE amount > 0 -- Exclude reversals/credits (business rule)
```

Step 3: Add tests:

```
yaml

tests:
  - name: stg_orders_no_negative_amounts
    description: "Revenue models only include positive amounts (business rule)"
    query: >
      SELECT COUNT(*) as negative_orders
      FROM {{ ref('stg_orders') }}
      WHERE amount < 0
```

Step 4: Reconciliation report:

```
sql

-- models/analysis/reconciliation_stg_vs_source.sql
SELECT
  'Source' as source,
  COUNT(*) as row_count,
  SUM(amount) as total_amount
FROM {{ source('salesforce', 'orders') }}

UNION ALL

SELECT
  'dbt Model',
  COUNT(*),
  SUM(amount)
FROM {{ ref('stg_orders') }}
```

Run monthly: `dbt run-operation generate_reconciliation_report`

40. How do you recover from a failed dbt run?

Answer:

Scenario: dbt run failed halfway, some models built, some didn't

Step 1: Check status:

```
bash

dbt logs # See which model failed and why
# Output: Model 'fct_orders' failed with SQL error
```

Step 2: Determine the issue:

```
bash

dbt compile --select fct_orders --debug

# Check generated SQL in target/compiled/
cat target/compiled/models/marts/fct_orders.sql
```

Step 3: Fix the issue:

```
sql

-- Example: Missing column in stg_orders
-- Fix: Add the column to staging model
SELECT
  order_id,
  customer_id,
  amount,
  status, -- Added this column
FROM {{ ref('stg_orders') }}
```

Step 4: Resume from failure point:

```
bash

dbt run --select fct_orders+ # Re-run failed model + downstream

# Or, re-run everything to be safe:
dbt run --full-refresh
```

Step 5: Verify data integrity:

```
bash

dbt test --select fct_orders+ # Test recovered models
dbt source freshness           # Ensure source is fresh
```

Prevention:

- Enable pre-hooks to validate source data
- Add dbt tests to catch issues early

- Use `dbt test` in CI before production deploy
-

41. How do you migrate a model from one materialization to another?

Answer:

Scenario: stg_customers is a VIEW (slow), need to convert to TABLE

Step 1: Change config:

```
sql

-- models/staging/stg_customers.sql
{{ config(materialized='table') }} -- Changed from 'view'

SELECT * FROM {{ source('crm', 'customers') }}
```

Step 2: Full refresh:

```
bash

dbt run --full-refresh --select stg_customers
# Drops old view, creates new table
```

Step 3: Monitor performance:

```
bash

dbt metrics report # Compare query times
# Before: 30 sec (view, recomputes on every query)
# After: 2 sec (table, pre-computed)
```

Step 4: Update downstream dependencies:

```
bash

dbt run --select stg_customers+ # Refresh models that depend on it
```

Revert if needed:

```
sql

{{ config(materialized='view') }}
dbt run --full-refresh --select stg_customers
```

42. How do you handle changing source schema in dbt?

Answer:

Scenario: Source table adds new column 'phone_number'

Without `on_schema_change` (breaks):

```
dbt run → SQL: SELECT * FROM {{ source(...) }}  
→ Succeeds  
→ But new column 'phone_number' not included  
→ Tests fail because column missing in downstream models
```

With `on_schema_change: 'sync_all_columns'` (auto-adapts):

```
yaml  
  
# models/staging/stg_customers.sql  
{{ config(  
    materialized='incremental',  
    on_schema_change='sync_all_columns'  
) }}  
  
SELECT * FROM {{ source('crm', 'customers') }}
```

dbt automatically:

1. Detects new column 'phone_number'
2. Adds column to existing table
3. Re-compiles downstream models
4. All tests pass ✅

Options:

- `ignore`: Do nothing (backwards compatible)
- `fail`: Stop dbt run, alert team
- `sync_all_columns`: Auto-add new columns (risky if source has garbage data)
- `append_new_columns`: Only add new, never remove old

Best practice: Use `append_new_columns` (safe) + manual review for removals.

43. How do you document dbt models effectively?

Answer:

Complete documentation example:

yaml

```
# models/_models.yml
version: 2

models:
  - name: fct_customer_revenue
    description: >
      Daily revenue by customer. One row per customer per day.
      Includes all orders with amount > 0.

    meta:
      owner: analytics-team
      priority: critical
      sla_minutes: 120
      contains_pii: false

    columns:
      - name: customer_id
        description: Unique customer identifier (PK)
        tests:
          - unique
          - not_null
        meta:
          data_type: integer

      - name: revenue_date
        description: Date of revenue (ISO 8601 format, UTC timezone)
        tests:
          - not_null
        meta:
          data_type: date

      - name: daily_revenue
        description: Sum of all order amounts for customer on that day
        tests:
          - dbt_expectations.expect_column_values_to_be_positive
        meta:
          data_type: decimal
          business_definition: "Sum of (order_amount) for orders WHERE amount"

      - name: customer_count_reached_goal
        description: TRUE if customer hit $100k lifetime revenue by this date
        meta:
          data_type: boolean
```

Code-level documentation:

```
sql

-- models/marts/fct_customer_revenue.sql
{{ config(
    materialized='table',
    description='Fact table of daily customer revenue'
) }}

-- Revenue fact table, updated daily
-- Filters: amount > 0 (excludes refunds/reversals)
-- Grain: one row per (customer_id, revenue_date)

SELECT
    c.customer_id,
    DATE(o.order_date) as revenue_date,
    SUM(o.amount) as daily_revenue -- Excludes refunds
FROM {{ ref('stg_customers') }} c
INNER JOIN {{ ref('stg_orders') }} o USING(customer_id)
WHERE o.amount > 0 -- Business rule: exclude negative amounts
GROUP BY 1, 2
```

Generate docs:

```
bash

dbt docs generate
dbt docs serve # View at http://localhost:8000
```

44. How do you test complex business logic in dbt?

Answer:

Example: Customer lifetime value with tiered discounts

```
sql
```

```
-- models/marts/fct_customer_ltv.sql
SELECT
  c.customer_id,
  SUM(o.amount) as lifetime_revenue,
  COUNT(o.order_id) as order_count,
  -- Tiered discount logic
  CASE
    WHEN COUNT(o.order_id) >= 100 THEN SUM(o.amount) * 0.95
    WHEN COUNT(o.order_id) >= 50 THEN SUM(o.amount) * 0.90
    WHEN COUNT(o.order_id) >= 10 THEN SUM(o.amount) * 0.85
    ELSE SUM(o.amount)
  END as lifetime_revenue_after_discount
FROM {{ ref('stg_customers') }} c
LEFT JOIN {{ ref('stg_orders') }} o USING(customer_id)
GROUP BY 1
```

Test the logic:

yaml

```
tests:
  - name: test_ltv_discount_tiers
    description: "Verify discount tiers apply correctly"
    query: >
      WITH customer_tiers AS (
        SELECT
          customer_id,
          order_count,
          CASE
            WHEN order_count >= 100 THEN 0.95
            WHEN order_count >= 50 THEN 0.90
            WHEN order_count >= 10 THEN 0.85
            ELSE 1.0
          END as expected_multiplier,
          lifetime_revenue_after_discount / lifetime_revenue as actual_multiplier
        FROM {{ ref('fct_customer_ltv') }}
      )
      SELECT *
      FROM customer_tiers
      WHERE ABS(expected_multiplier - actual_multiplier) > 0.001
      -- If query returns rows, there's a discount calculation error
```

Property-based testing (dbt-expectations):

yaml

```
columns:
  - name: lifetime_revenue_after_discount
tests:
  - dbt_expectations.expect_column_values_to_be_of_type:
      column_type: decimal
  - dbt_expectations.expect_column_values_to_be_between:
      min_value: 0
      max_value: 10000000
  - dbt_expectations.expect_column_values_to_be_less_than_or_equal_to:
      value: lifetime_revenue
      column_name: lifetime_revenue
```

45. How do you implement data quality SLAs in dbt?

Answer:

Define SLA:

```
yaml
# models/_models.yml
models:
  - name: fct_daily_revenue
    meta:
      owner: finance-team
      sla:
        freshness_minutes: 60          # Must be latest by 1 hour
        completeness_pct: 95          # ≥95% of rows loaded
        accuracy_threshold: 0.99      # ≥99% accurate vs source
        latency_minutes: 30           # Ready within 30 min of data landing
```

Implement freshness checks:

```
yaml
sources:
  - name: raw
    tables:
      - name: orders
        loaded_at_field: created_at
        freshness:
          warn_after: {count: 30, period: minute}
          error_after: {count: 60, period: minute}
```

Implement completeness checks:

```

sql

-- tests/assert_daily_revenue_completeness.sql
WITH expected_rows AS (
    SELECT COUNT(*) as count FROM {{ source('orders', 'raw_orders') }}
    WHERE DATE(created_at) = CURRENT_DATE
),

actual_rows AS (
    SELECT COUNT(*) as count FROM {{ ref('fct_daily_revenue') }}
    WHERE DATE(revenue_date) = CURRENT_DATE
)

SELECT *
FROM actual_rows
WHERE count < (SELECT count * 0.95 FROM expected_rows)
-- Alert if < 95% of expected rows loaded

```

Implement latency tracking:

```

sql

-- Post-hook in model
INSERT INTO analytics.sla_metrics (model_name, load_timestamp, latency_minute)
SELECT
    '{{ this.name }}',
    CURRENT_TIMESTAMP,
    DATEDIFF(MINUTE, MAX(source_timestamp), CURRENT_TIMESTAMP) as latency
FROM {{ source('raw', 'orders') }}

```

Monitor SLAs:

```

bash

dbt source freshness          # Check freshness
dbt test --select severity:error # Check data quality
# If any fail → SLA breach → Alert team

```

Advanced Topics

46. How do you handle recursive relationships in dbt?

Answer:

Example: Employee table with manager_id (self-referential)

employee_id	name	manager_id
1	Alice	NULL
2	Bob	1
3	Charlie	2

Query: "Find all reports under Alice"

Using CTE recursion:

```
sql

-- models/marts/org_hierarchy.sql
WITH RECURSIVE employee_hierarchy AS (
  -- Base case: Start with top-level employees
  SELECT
    employee_id,
    manager_id,
    name,
    0 as level
  FROM {{ ref('stg_employees') }}
  WHERE manager_id IS NULL

  UNION ALL

  -- Recursive case: Find direct reports
  SELECT
    e.employee_id,
    e.manager_id,
    e.name,
    eh.level + 1
  FROM {{ ref('stg_employees') }} e
  INNER JOIN employee_hierarchy eh
    ON e.manager_id = eh.employee_id
  WHERE eh.level < 10 -- Prevent infinite recursion
)

SELECT *
FROM employee_hierarchy
ORDER BY level, employee_id
```

Test the hierarchy:

yaml

```

tests:
- name: test_org_hierarchy_no_cycles
  description: "Prevent circular manager relationships"
  query: >
    WITH RECURSIVE cycle_check AS (
      SELECT 1 as depth, employee_id, manager_id
      FROM {{ ref('stg_employees') }}
      WHERE manager_id IS NOT NULL

      UNION ALL

      SELECT depth + 1, c.employee_id, e.manager_id
      FROM cycle_check c
      JOIN {{ ref('stg_employees') }} e
        ON c.manager_id = e.employee_id
      WHERE depth < 20
    )
    SELECT employee_id FROM cycle_check
    WHERE depth = 20 -- Depth limit reached = cycle detected

```

47. How do you handle multi-currency transactions in dbt?

Answer:

Problem: Orders in USD, EUR, GBP; need to aggregate as single currency

Solution: Exchange rate lookup:

```

sql

-- models/staging/stg_exchange_rates.sql
SELECT
  currency,
  exchange_date,
  rate_to_usd
FROM {{ source('treasury', 'daily_exchange_rates') }}

```

Apply in fact table:

```

sql

```



```
-- models/marts/fct_orders_normalized.sql
SELECT
  o.order_id,
  o.currency,
  o.amount as amount_in_original_currency,
  o.amount * er.rate_to_usd as amount_in_usd,
  o.order_date,
  er.exchange_date
FROM {{ ref('stg_orders') }} o
LEFT JOIN {{ ref('stg_exchange_rates') }} er
  ON o.currency = er.currency
  AND DATE(o.order_date) = er.exchange_date
```

Test for missing rates:

```
yaml

tests:
  - name: test_no_missing_exchange_rates
    query: >
      SELECT *
      FROM {{ ref('fct_orders_normalized') }}
      WHERE rate_to_usd IS NULL
      -- Alert if any orders missing exchange rate
```

Document assumptions:

```
sql

-- Exchange rate from 2025-06-16
-- If order timestamp is 2025-06-16 13:30 UTC,
-- uses rate from 2025-06-16 close-of-business (EOD)
```

48. How do you implement A/B test analysis in dbt?

Answer:

Example: Testing new checkout flow

```
sql
```

```
-- models/analysis/ab_test_checkout_conversion.sql
WITH test_groups AS (
  SELECT
    user_id,
    'control' as test_group
  FROM {{ ref('stg_users') }}
  WHERE user_cohort_date = '2025-01-01'
    AND RAND() < 0.5 -- 50% to control

  UNION ALL

  SELECT
    user_id,
    'treatment' as test_group
  FROM {{ ref('stg_users') }}
  WHERE user_cohort_date = '2025-01-01'
    AND RAND() >= 0.5 -- 50% to treatment
),

checkout_metrics AS (
  SELECT
    tg.test_group,
    COUNT(DISTINCT tg.user_id) as users,
    COUNT(DISTINCT o.order_id) as conversions,
    COUNT(DISTINCT o.order_id) / COUNT(DISTINCT tg.user_id) as conversion_rate,
    AVG(o.amount) as avg_order_value,
    SUM(o.amount) as total_revenue
  FROM test_groups tg
  LEFT JOIN {{ ref('stg_orders') }} o
    ON tg.user_id = o.customer_id
    AND o.order_date >= '2025-01-01'
    AND o.order_date < '2025-02-01'
  GROUP BY 1
)

SELECT *
FROM checkout_metrics
```

Statistical significance test:

sql

```

-- models/analysis/ab_test_significance.sql
-- Use two-sample t-test to compare means
WITH test_metrics AS (
  -- Calculated above
),

stats AS (
  SELECT
    test_group,
    COUNT(*) as n,
    AVG(conversion_rate) as mean,
    STDDEV(conversion_rate) as stddev
  FROM test_metrics
  GROUP BY test_group
)

SELECT
  SQRT(
    (c.stddev^2 / c.n) + (t.stddev^2 / t.n)
  ) as standard_error,
  (c.mean - t.mean) / SQRT(
    (c.stddev^2 / c.n) + (t.stddev^2 / t.n)
  ) as t_statistic,
  CASE
    WHEN ABS((c.mean - t.mean) / SQRT((c.stddev^2 / c.n) + (t.stddev^2 / t.n))
    THEN 'Significant (p < 0.05)'
    ELSE 'Not significant'
  END as result
FROM stats c, stats t
WHERE c.test_group = 'control'
AND t.test_group = 'treatment'

```

49. How do you handle dbt run failures due to resource limits?

Answer:

Problem: Redshift cluster runs out of memory during `dbt run`

Solutions:

1. Reduce parallel threads:

yaml

```
# profiles.yml
my_analytics:
  outputs:
    prod:
      threads: 4 # Reduce from 8 to 4 (less concurrent queries)
```

2. Materialize intermediates as tables (not ephemeral):

```
sql

{{ config(materialized='table') }} # Use table to avoid huge CTEs
```

3. Increase warehouse resources:

```
sql

-- Redshift
ALTER CLUSTER my_cluster MODIFY (node_count 5); -- Scale up

-- Snowflake
ALTER WAREHOUSE analytics SET WAREHOUSE_SIZE = 'XL'; -- Bigger warehouse

-- Athena (auto-scales, but may increase query costs)
```

4. Use resource groups (Redshift):

```
sql

-- Limit dbt jobs to 40% of cluster resources
CREATE RESOURCE GROUP dbt_group WITH (query_execution_time_limit=3600000);
GRANT RESOURCE_GROUP dbt_group TO dbt_user;
```

5. Break dbt run into stages:

```
bash

dbt run --select staging/           # Run staging first (small)
dbt run --select intermediate/     # Then intermediate
dbt run --select marts/            # Finally marts (large)
```

50. How do you implement dbt best practices in your organization?

Answer:

1. Establish coding standards:

- Naming: snake_case, prefix with stg_/int_/fct_/dim_
- Column names: lowercase, descriptive, no abbreviations
- Tests: Every PK column needs unique + not_null
- Documentation: Every model + every column must have description
- Linting: Use dbt-sqlfluff to auto-enforce style

2. Setup dbt project template:

```
bash

# Create template with proper structure
dbt-init-template --organization acme

# All teams fork and customize
```

3. Code review process:

```
PR created → dbt Cloud runs slim CI
  → Code review (1+ approvals)
  → Merge to main
  → Production job runs
```

4. Training & documentation:

- Wiki: dbt style guide, common patterns
- Office hours: Weekly dbt help
- Templates: Copy-paste models (stg_, int_, marts/*)

5. Monitor dbt health:

```
sql

-- Track slow models
SELECT
  model_name,
  AVG(execution_seconds) as avg_time,
  MAX(execution_seconds) as max_time,
  COUNT(*) as runs
FROM dbt_metrics
GROUP BY 1
ORDER BY avg_time DESC
-- Alert if any model > 10 min
```

6. Version dbt consistently:

```
yaml
```

```
# dbt_project.yml
require-dbt-version: [ '>=1.4.0', '<1.6.0' ] # Lock version
```

7. Secure credentials:

```
bash

# Use dbt Cloud secrets or GitHub Actions secrets
# Never commit profiles.yml
git update-index --skip-worktree ~/.dbt/profiles.yml
```

8. Continuous improvement:

- Monthly dbt office hours
 - Quarterly architecture reviews
 - Encourage experimentation with new features
 - Share learnings across teams
-

Summary

These 50 questions cover everything from dbt fundamentals to advanced real-world scenarios. For staff-level interviews, focus on:

1. **Architecture decisions** (why materialization choices)
2. **Performance optimization** (how to scale dbt)
3. **Data governance** (PII, testing, monitoring)
4. **CI/CD & deployment** (how dbt fits in pipeline)
5. **Real-world problem-solving** (recovery, debugging, schema changes)

Good luck with your interviews!